

PRESENTER NOTES

Generalized Transformers

A fifteen-minute talk for a predictive-coding audience

Companion to the 14-slide deck and to the report *Generalized Transformers: Geometry, Compression, Predictive Coding, and Continual Learning*, September 2026

How to use these notes

There is one section per slide. Each begins with a reminder of what is on the slide and a suggested way of talking it through. Then every term and idea that appears on the slide is explained twice: first in plain words, the way one might explain it to a first-year computing student, and then, after the marker **IN SYMBOLS**, in the notation of the report. Each section ends with questions the audience is likely to ask, and short answers. You will not say most of this aloud. It is here so that whatever a listener asks about a slide, the answer is on the page for that slide.

All numbers are taken from the report. Losses are in bits per byte (bpb) on held-out text, lower is better, and unless stated otherwise they are means over five random seeds that share one frozen base model, one corpus and one dictionary.

Timing

Slide	Topic	Min.	Slide	Topic	Min.
1	Title	0.5	8	The energy view	1
2	What a fixed architecture leaves out	1	9	H2, the working cap	2
3	The core claim	0.5	10	Where the gain comes from	1.5
4	One layer, six jobs	2	11	What PC did and did not do	1.5
5	The binding example	1	12	Forgetting	1
6	What a compressed state must keep	1	13	Open problems	1.5
7	Three ways to respond to an error	1	14	Close	0.5

That is fifteen minutes with nothing to spare. If you are running late, slide 5 can be told in thirty seconds and slide 8 can be shown without comment to this audience.

The argument in five sentences

A transformer layer does six separable jobs, and attention with an MLP is one way of doing them. One of the six, reconciling conflicting evidence, is left empty in a standard transformer, and it is the natural home of predictive-coding inference. Compression-driven distinction (CD) adds a slow process that changes the structure of the network, beside PC's fast inference and its slower weight learning. A small system of this kind, attached as a "cap" to a frozen transformer, improves prediction substantially; most of the gain comes from a richer memory, and smaller, consistent gains come from structure search and from settling. As a learning rule PC ties with backpropagation so far, and the questions that would decide whether PC becomes central to this program are ones this audience is equipped to answer.

Slide 1 about 30 seconds**Title**

On the slide. Kicker: “A research program, and where predictive coding fits in it.” Title: *Generalized Transformers*. Subtitle: “Compression-driven structure, settling as inference, and caps that keep learning after deployment.”

What to say

“I am going to describe a way of looking at transformers that has an empty slot in it, and the slot has the shape of predictive coding. Then I will show you a small working system built on that view, give you the numbers without dressing them up, including where PC helped and where it did not, and finish with a list of open problems that I think people in this room are better placed to solve than we are.”

The subtitle lists the three ingredients in the order in which the talk takes them up, and each phrase is unpacked below so that you can gloss any of them if asked at once.

Terms and concepts

Generalized transformer (GT). A description of what any transformer-like layer does, broken into six jobs: work out what is needed, choose where to look, bring back evidence, compute helpful features, reconcile conflicts, and write the next state. The ordinary transformer is one way of filling in the six jobs. Systems that look things up in dictionaries, run small programs, or perform iterative inference are other ways.

IN SYMBOLS. A GT layer maps the state x_i at position i to

$$x_i^+ = R\left(\mathcal{S}\left(\Phi\left[x_i, \left\{\sum_j a_{ij} \delta_{(j, V(x_j))}\right\}\right]\right)\right),$$

where a_{ij} are routing weights, V produces payloads, Φ adds features, $\mathcal{S}$ is an optional inference step and R re-encodes. Slide 4 explains every symbol.

Compression-driven structure. The network’s structure, meaning which features and memory readers it contains, is not fixed in advance. A search process proposes changes and keeps those that explain the data better for their cost. “Compression” is the yardstick: a good explanation lets you write the data down more briefly.

IN SYMBOLS. Structure M and parameters θ are chosen to reduce a description-length objective of the form $L(M, \theta) + L(D \mid M, \theta)$ plus charges for running and searching, subject to explicit budgets. See slide 6.

Settling as inference. Before it answers, the network spends a few steps adjusting some temporary internal numbers so that they agree as well as possible both with what it expected and with the evidence in front of it. This is the inference phase of predictive coding, and the audience knows it well. The claim of the talk is that this step has a definite place in a transformer-like layer.

IN SYMBOLS. Given an energy $E_0(z; x)$, settling is $z^{k+1} = z^k - \eta \nabla_z E_0(z^k; x)$ for K steps, and the prediction is read from z^K . In the GT equation this is the map $\mathcal{S}$.

Cap. An add-on module that sits on top of an existing, frozen transformer. It reads the transformer's internal state and its prediction, consults extra memories, and outputs a corrected prediction. Because the base already works, the new machinery only has to improve on it.

IN SYMBOLS. With base distribution $q_0 = p_B$ and further readers $q_1, \dots, q_n$, the cap outputs $p_C(y | x) = \sum_j w_j(x) q_j(y | x)$ with $w(x)$ on the probability simplex.

Keep learning after deployment. Ordinary models are trained, then frozen and used. The aim here is a system that can go on acquiring knowledge while in use, without wrecking what it already knows. Slide 12 reports a first measurement of how far the prototype is from that.

Questions you may get

Is this a replacement for transformers? No claim of that kind is made. The working system is a small attachment to a transformer. Whether the ideas could eventually replace parts of the dense network is left as an empirical question for much later (slide 14).

Slide 2 about 1 minute**What fitting a fixed architecture leaves out**

On the slide. Five numbered rows: continual learning; learning during use; exchange of capabilities; neural-symbolic links; creativity off-distribution. Each has a one-line description.

What to say

“Today’s models are fitted in one phase and used in another, with the weights frozen in between. That split is why they work so well, and it is also why five things we would like are awkward. None of them is impossible for a transformer. The question is whether an architecture that is explicit about what it preserves, how new modules are found, and how adaptation is kept local makes them easier to combine and to measure. This talk has evidence on the first two and proposals for the rest.”

Do not linger. Read the five names, give one sentence to the first, and move on.

Terms and concepts

Fitting a fixed architecture. Choosing the numbers inside a network whose wiring has been decided beforehand. Training changes the numbers and never the wiring.

IN SYMBOLS. $\theta^* = \arg \min_{\theta} \mathbb{E}_{(x,y)} [-\log p_{\theta}(y | x)]$ for a fixed function family $\{p_{\theta}\}$. In a CD system the family itself, indexed by a structure M , is also searched.

Continual learning; stored versus reachable knowledge. Learning something new without losing the old. A frozen model keeps its old skill only by learning nothing, so it does not count. A perfect archive does not count either if the system can no longer find the right entry. The slide separates *having* knowledge from *reaching* it, and slide 12 shows a failure of the second kind.

IN SYMBOLS. For an old domain A and new domain B , with checkpoints θ_A before and $\theta_{A \rightarrow B}$ after adaptation,

$$F_A = L_A(\theta_{A \rightarrow B}) - L_A(\theta_A) \text{ (forgetting)}, \quad G_B = L_B(\theta_A) - L_B(\theta_{A \rightarrow B}) \text{ (new learning)}.$$

Both must be reported, since a frozen model has $F_A = G_B = 0$.

Learning during use; timescales. Four things should be able to go on together: answering, briefly adjusting internal state, updating weights, and discovering new structure. They run at different speeds, like a shop that serves customers all day, restocks each evening and refits the premises once a year, without closing.

IN SYMBOLS. Fast: $z \leftarrow \text{Settle}_K(E_0, z^0)$ per token. Middle: $\theta \leftarrow \theta - \eta \widehat{\nabla}_{\theta} C$ after outcomes. Slow: $M \leftarrow M'$ after a search episode. A versioned predictor is published atomically so that a running prediction always sees one coherent (M, θ) .

Module; exchange of capabilities. A skill packaged so that it can be handed to another system, which can test it before using it. In today’s models a skill is smeared across millions of weights and cannot be copied out.

IN SYMBOLS. A proposed module record is

$$\mathbf{m} = (\text{typed ports, program or weights, version, dependencies,} \\ \text{evidence, uncertainty, resource profile}).$$

A recipient accepts $\mathbf{m}$ only if its own measured $G_B > 0$ and F_A stays within a tolerance.

Neural-symbolic links; typed evidence. A logic engine or a program should be able to put a fact, a probability or a rule into the same workspace that the neural part reads, labelled with what kind of thing it is.

IN SYMBOLS. In GT terms a symbolic system is one more *source* j with payload $V(x_j)$, for instance a distribution $q_j(y | x)$ together with its provenance. It enters the layer through the same routing weights a_{ij} as any other source.

Creativity off-distribution; composition. When the familiar computations fail, build a new one out of known parts and check that it works on cases not seen before. Novelty alone does not count. The candidate has to survive a test.

IN SYMBOLS. Given a grammar $\mathcal{G}$ of operators, the test holds out whole *compositions* $g_1 \circ g_2$ during training and asks whether search recovers them, scored on fresh instances. This tests new compositions within $\mathcal{G}$ and cannot show the invention of new primitives.

Questions you may get

Doesn't retrieval or LoRA already do this? They address parts of it, and the report says so. The narrower question here is whether one explicit architecture makes the five goals easier to combine and to measure together.

Slide 3 about 30 seconds**The core claim**

On the slide. “A transformer layer does six jobs. Attention plus an MLP is one way of doing them. Name the jobs and the alternatives become design choices: lookups, small programs, explicit records, and an inference step that settles.”

What to say

Read the first two sentences and pause. Then: “Everything that follows comes from taking those six jobs seriously as separate interfaces. Once they are separate, you can swap in a dictionary lookup for attention, or keep records instead of averaging them, or, the part that concerns this room, put an inference step where the standard transformer has none.”

Terms and concepts

Attention. Each position in the text asks “who has what I need?”, scores every earlier position for relevance, turns the scores into weights that sum to one, and takes the weighted average of what those positions offer.

IN SYMBOLS. $m_i = \sum_j \text{softmax}_j(q_i^\top k_j / \sqrt{d_k}) v_j$, with $q_i = W_Q x_i$, $k_j = W_K x_j$, $v_j = W_V x_j$, and $j \leq i$.

MLP. A small two-layer network applied separately at each position after attention. It computes new features from what attention brought back.

IN SYMBOLS. $x_i^+ = N_2(u_i + W_2 \sigma(W_1 u_i + b_1) + b_2)$, where $u_i = N_1(x_i + W_O m_i)$ and N_1, N_2 are normalizations.

Lookups, small programs, explicit records. Three alternatives to soft attention. A lookup finds the exact matching entry in a table. A small program computes an answer by a short explicit procedure, such as “find the most recent earlier occurrence of this phrase and return what followed it.” An explicit record keeps an item whole, with a note of where it came from, where attention would blend it into an average.

IN SYMBOLS. A lookup is routing with a point mass: $a_{ij} = 1$ for the matching j and 0 elsewhere. A program is a payload map V or feature map g given by executable code. A record is an atom $(j, V(x_j))$ of the tagged measure on slide 4, left unaveraged.

An inference step that settles. A short iterative computation, carried out before the layer commits to an output, that adjusts temporary state until expectations and evidence agree as well as they can. This is the map $\mathcal{S}$. In a standard transformer it is the identity.

Questions you may get

Isn't “six jobs” just a relabelling of any function? It has content because it imposes restrictions. All access between positions must be declared through the routing weights and payloads, the local maps see only that evidence, the order of observation is explicit, and the costs of reading, storing, inferring and rewriting are counted separately.

Slide 4 about 2 minutes

One layer, six jobs

On the slide. Six boxes in a row, joined by arrows: Need, Route, Transport, Lift, Reconcile (highlighted), Re-encode. Below them the layer equation. Below that: a standard block has jobs 1–3 as attention with an immediate average, job 4 as the MLP, job 6 as residual plus norm, and job 5 as the identity.

What to say

“Think of each position in the text as a small desk with some notes on it. In one layer, the desk first decides what it needs. It picks which other desks to consult. It brings back what they have, keeping a note of who said what. It works out some helpful derived facts. If its sources disagree, it takes a moment to reconcile them. Then it writes up a fresh set of notes for the next layer.

In a standard transformer, the first three are attention, which averages what it fetched immediately, the fourth is the MLP, and the last is the residual connection and layer norm. The fifth job, the highlighted one, is skipped. It is the identity map. That empty slot is where predictive-coding inference goes.”

Then point at the equation and read it from the inside out: “route and fetch, put it beside what I already have, add features, settle, re-encode.”

Terms and concepts

Need (job 1). The desk writes down what it is looking for.

IN SYMBOLS. A query $q_i = Q(x_i)$ computed from the current state. In a transformer Q is the linear map W_Q .

Route (job 2); permitted sources; weights that sum to one. The desk has a fixed amount of attention to hand out among the sources it is allowed to read. It may not read the future, and in particular it may not read the token it is about to predict. How it divides the attention is the routing rule.

IN SYMBOLS. For each position i there is a permitted set $J_i \subseteq \{1, \dots, i\}$ and weights

$$a_{ij} \geq 0, \quad \sum_{j \in J_i} a_{ij} = 1, \quad a_{ij} = 0 \ (j \notin J_i).$$

Row i is a probability distribution over sources, and the table (a_{ij}) is called a *coupling*. Columns need not sum to one. Softmax attention is the solution of a small optimization: given costs c_{ij} , a prior π_i and temperature $\tau > 0$,

$$a_i^* = \arg \min_{a \in \Delta(J_i)} \sum_j a_j c_{ij} + \tau \text{KL}(a \| \pi_i) \implies a_{ij}^* = \frac{\pi_i e^{-c_{ij}/\tau}}{\sum_r \pi_i e^{-c_{ir}/\tau}}.$$

The proof is one line: the objective equals $\tau \text{KL}(a \| a^*)$ plus a constant. With uniform π_i , $c_{ij} = -q_i^\top k_j / \sqrt{d_k}$ and $\tau = 1$ this is scaled dot-product attention. A PC audience will recognize the form: expected cost plus a KL term is a free energy.

Transport (job 3); tagged evidence. The desk brings back what its sources offer and keeps the labels on. Think of a folder of index cards, each saying “from desk j , weight a_{ij} , content v_j .” Standard attention tips the folder into a blender at once. The framework asks you to notice that the blender is optional.

IN SYMBOLS. The incoming evidence is the measure

$$\nu_i = \sum_{j \in J_i} a_{ij} \delta_{(j, V(x_j))},$$

where $\delta_{(j,v)}$ is a unit point mass on the pair (source j , payload v). Standard attention replaces ν_i by its barycenter $B(\nu_i) = \sum_j a_{ij} V(x_j)$. The measure keeps which items were selected, with what weight and from where. It cannot restore anything that V has already discarded.

Lift (job 4); “nothing is dropped”. The desk adds some derived facts that make later work easier, without throwing away what it had. A database index is the right comparison, since it adds no facts to the database and changes what can be done quickly.

IN SYMBOLS. With $z_i = (x_i, \nu_i^1, \dots, \nu_i^H)$ for H heads, the lift is $\Phi_G(z_i) = (z_i, g_1(z_i), \dots, g_m(z_i))$. Its first coordinate is z_i itself, so Φ_G is injective. The features g_k add no information and change access cost.

Reconcile (job 5); the map $\mathcal{S}$. If sources disagree, or if recent experience says one source has been unreliable, the desk takes a few moments to adjust how much it trusts each before answering. It may only use what has already been observed.

IN SYMBOLS. $\mathcal{S}$ is a finite inference procedure, by default the identity. In the energy-based realization, $\mathcal{S}(z^0) = z^K$ where $z^{k+1} = z^k - \eta \nabla_z E_0(z^k; x)$ and E_0 contains prediction-error terms and evidence terms available at prediction time (slide 8).

Re-encode (job 6). The desk writes the notes that the next layer will see. It cannot keep everything, so it should keep what later computations will need. Slide 6 makes “what they will need” precise.

IN SYMBOLS. $x_i^+ = R(\cdot)$. In a transformer R is the projection, residual sum and normalization. In an ideal CD layer R is a behavioral quotient Q_C .

The layer equation.

IN SYMBOLS.

$$x_i^+ = R\left(\mathcal{S}\left(\Phi_G\left[x_i, \left\{\sum_j a_{ij}^h \delta_{(j, V_h(x_j))}\right\}_{h=1}^H\right]\right)\right).$$

The slide shows one head for readability.

The standard block as a special case.

IN SYMBOLS. Take Q, K, V linear, a the softmax coupling, aggregate each ν_i^h by its barycenter, let the features g_k be the intermediate quantities u_i and $\sigma(W_1 u_i + b_1)$, set $\mathcal{S} = \text{id}$, and let R return $N_2(u_i + W_2 \sigma(W_1 u_i + b_1) + b_2)$. Substituting into the layer equation reproduces the transformer block line by line, so the containment is exact.

Questions you may get

Is job 5 related to iterative or looped transformers, or deep equilibrium models? Yes, those are other ways of filling the same slot. The point of naming it is that the PC literature offers a principled choice of what to iterate: an energy that balances a learned prior against observed evidence.

Why measures? Isn't that heavy machinery for a weighted sum? It is only a way of writing “the weighted list before it was summed.” Its use is to make visible the moment at which information is given up, which the next slide illustrates.

Slide 5 about 1 minute**Averaging too early loses the binding**

On the slide. The sentence “The captain is Mira. The cook is Leo. The captain is ____.” Two memory cards: A (captain–Mira, cook–Leo) and B (captain–Leo, cook–Mira). A third card: the uniform average of the records is the same for A and B. Bottom line: route selectively, or lift a joint feature before aggregating.

What to say

“Here is why the order of those jobs is not a technicality. Two ships have the same two jobs and the same two people, with the roles swapped. Write each fact as ‘role plus name’ and average the facts. The two ships now look identical. You know there is a captain and a cook, a Mira and a Leo, and you have lost who is which. Nothing you compute afterwards can get it back, because the two inputs have become the same input. If you look up only the captain’s record, or compute a joint role-and-name feature before averaging, the information survives. I am not claiming transformers fail at this. Several heads can solve it. I am showing that where information gets thrown away is a design decision, and the framework makes you take it in the open.”

Terms and concepts

Binding. Knowing which things go together: that Mira goes with captain and Leo with cook. It is distinct from knowing which things are present.

One-hot code. The simplest way to turn a category into numbers: a list of zeros with a single one marking the category.

IN SYMBOLS. Roles: captain = e_1 , cook = e_2 in $\mathbb{R}^2$. Names: Mira = e_1 , Leo = e_2 in $\mathbb{R}^2$. A record is the concatenation $[\text{role}; \text{name}] \in \mathbb{R}^4$.

Uniform average of the records.

IN SYMBOLS. Memory A holds $[e_1; e_1] = (1, 0, 1, 0)$ and $[e_2; e_2] = (0, 1, 0, 1)$, whose average is $\frac{1}{2}(1, 1, 1, 1)$. Memory B holds $[e_1; e_2] = (1, 0, 0, 1)$ and $[e_2; e_1] = (0, 1, 1, 0)$, whose average is also $\frac{1}{2}(1, 1, 1, 1)$. For any function f , however nonlinear, $f(\text{avg}_A) = f(\text{avg}_B)$.

“No later nonlinearity can bring it back”. A function gives the same output on the same input. Once two situations have been mapped to one vector, no downstream cleverness can separate them. In the language of slide 6, the average has merged two states that a later question (“who is the captain?”) distinguishes.

Route selectively. Read only the captain’s record.

IN SYMBOLS. Put all routing weight on the matching source: $a_{i, \text{captain}} = 1$. The payload returned is $[e_1; e_1]$ under A and $[e_1; e_2]$ under B, which differ.

Lift a joint feature before aggregating. Compute a “role and name together” feature for each record first, then average.

IN SYMBOLS. Use the outer product role name $\top \in \mathbb{R}^{2 \times 2}$. Under A the average is $\frac{1}{2}(e_1 e_1^\top + e_2 e_2^\top) = \frac{1}{2} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, and under B it is $\frac{1}{2} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. These differ, so the binding survives the averaging.

Weighted collection that remembers its sources; barycenter. The tagged measure ν_i of slide 4, and its weighted mean $B(\nu_i)$. “A choice made in the open” means that the framework represents the collection first and treats the step of averaging it as an explicit design decision.

Questions you may get

Real transformers solve this task. So what is the point? They solve it because multiple heads and learned value maps amount to selective routing and joint features. The example is about the interface. It shows a specific way to lose information and the two general remedies, which are the ones a CD system uses by design.

Slide 6 about 1 minute**What a compressed state must keep**

On the slide. A formula for the behavioral distance d_C . A statement: two states may share one code exactly when no declared later computation can tell them apart. Three cards: the price of merging ($\delta/2$); precision from later use (the Fisher matrix G); CD as a GT layer (rules, exceptions, index, budgets).

What to say

“Job six was ‘keep what later computations need.’ Here is what that means exactly. Two situations can be filed under the same note if every later question we care about gets the same answer from either. If you file together two situations that some later question treats differently, that question must be answered wrongly for at least one of them, and I can tell you by how much. For this audience the middle card is the interesting one. If you ask how sensitive later predictions are to a small change in the state, you get a Fisher matrix, which is a precision matrix over directions in state space, set by downstream use. Nobody has yet put that into a PC energy. The last card is the definition of a CD network: a layer whose final step tries to approximate this ideal, using reusable rules, a list of exceptions to them, and an index, under a budget.”

Terms and concepts

State; code. The state is everything on the desk. The code is the shorter note written in its place.

IN SYMBOLS. A re-encoding is a map $r : Z \rightarrow U$ from states to codes, with $|U|$ typically much smaller than $|Z|$.

Continuation; “declared later computation”. Anything that will be done with the note later: answering a question, predicting the next token, calling a module, routing in the next layer. The set of these has to be written down in advance. Otherwise any compression could be called lossless by declaring only the uses that survive it.

IN SYMBOLS. A family $\mathcal{C}$, each $c \in \mathcal{C}$ giving an output distribution $p_c(\cdot | z)$.

Total variation (TV). A number between 0 and 1 that says how different two probability distributions are: the largest difference in the probability they assign to any event.

IN SYMBOLS. $\text{TV}(p, q) = \frac{1}{2} \sum_y |p(y) - q(y)| = \max_A |p(A) - q(A)|$.

Behavioral distance. How differently the most sensitive later computation would treat two states. If it is zero, nothing downstream can tell them apart.

IN SYMBOLS.

$$d_C(z, z') = \sup_{c \in \mathcal{C}} \text{TV}(p_c(\cdot | z), p_c(\cdot | z')).$$

This is a pseudometric: symmetric, satisfies the triangle inequality, and may be zero for $z \neq z'$.

“May share one code exactly when . . .”; fibers and the quotient. The set of states filed under one code is called a fiber. Filing is lossless exactly when the states within every fiber are behaviorally identical.

IN SYMBOLS. For finite Z : r preserves every $c \in \mathcal{C}$ exactly $\iff$ every fiber $r^{-1}(u)$ has $d_{\mathcal{C}}$ -diameter zero $\iff$ each p_c factors as $\hat{p}_c \circ r$. The coarsest such r is the behavioral quotient $Q_{\mathcal{C}}$, which merges precisely the pairs with $d_{\mathcal{C}} = 0$.

The price of merging; decoder. A decoder is whatever reads the code and tries to reproduce the later computation's output. If two states at distance δ share a code, the decoder sees the same input for both and must give one answer.

IN SYMBOLS. Let $\hat{p}$ be the decoder's single output for the shared code, and p, p' the true outputs with $\text{TV}(p, p') = \delta$. By the triangle inequality $\delta \leq \text{TV}(p, \hat{p}) + \text{TV}(\hat{p}, p')$, so at least one error is $\geq \delta/2$. Conversely, if every fiber has diameter $\leq \epsilon$, decoding by a representative of the fiber has error $\leq \epsilon$.

Fisher matrix; “precision from later use”. Ask: if I nudge the state a little in some direction, how much do later predictions change? Directions that change them a lot should be preserved carefully, and directions that change nothing can be compressed away. The matrix that records this is the Fisher information. In PC language it plays the part of a precision matrix, with the difference that it is set by downstream use instead of being learned from the statistics of the errors.

IN SYMBOLS. For smooth positive p_c and fixed weights ρ_c ,

$$G(z) = \sum_c \rho_c \sum_y \frac{\nabla p_c(y | z) \nabla p_c(y | z)^\top}{p_c(y | z)},$$

$$\sum_c \rho_c \text{KL}(p_c(\cdot | z) \| p_c(\cdot | z + \Delta)) = \frac{1}{2} \Delta^\top G(z) \Delta + o(\|\Delta\|^2).$$

The derivation expands the KL to second order; the linear term vanishes because probabilities sum to one. The suggestion on the card is to use G for the precisions Λ_v of slide 8. A zero direction of G shows insensitivity only to first order. This has not been implemented.

CD as a GT layer; rules, exceptions, index. A CD representation explains its data with reusable rules, stores separately whatever the rules get wrong, and keeps an index for finding the right pieces fast. “The captain is whoever was most recently assigned” is a rule. A sentence that breaks the rule goes on the exceptions list.

IN SYMBOLS. Data D are represented by a program graph M , parameters θ , features F , residual R_D and index I , with $D = \text{Decode}(M, \theta, F, R_D, I)$ in the lossless case. The ideal layer is $x_i^+ = Q_{\mathcal{C}_i}(\mathcal{S}(\text{Expose}_{G_i}[x_i \sqcup (A_i(X) \triangleright V(X))]))$, which is the GT layer with $R = Q_{\mathcal{C}}$.

Memory and work budgets.

IN SYMBOLS. The idealized objective is $J = L(M, \theta, I | \mathcal{L}) + L(D | M, \theta, I) + \lambda C_{\text{run}} + \gamma C_{\text{search}} + \mu \mathcal{D}_{\mathcal{C}}$: model length given a shared library $\mathcal{L}$, data length given the model, running cost, search cost and behavioral distortion. In practice candidates are chosen by predictive gain under hard ceilings on memory and work.

Questions you may get

Who chooses $\mathcal{C}$? The designer, and that is a real limitation. The guarantee covers only the declared uses. In practice $\mathcal{C}$ is a finite set of probes, replay examples and protected behaviors.

Is this the information bottleneck? It is closely related. The bottleneck trades compression against mutual information with one target variable. Here the target is a family of downstream computations, and the worst-case distance gives the exact $\delta/2$ statement.

Slide 7 about 1 minute**Three ways to respond to an error**

On the slide. Three cards: revise the answer (PC inference, fast, every token); revise the habit (PC local rule or BP, after outcomes); revise the machinery (CD structural search, slow, background). Bottom line: PC already separates the first two, CD adds the third, and all of it can sit in a cap on a frozen transformer.

What to say

“When a learner gets something wrong, it can change three different things. It can change its mind about this case: maybe the copy reader deserves more trust on this particular sentence. It can change a habit: maybe it should generally trust that reader in contexts like this one. Or it can change its equipment: maybe it needs a reader it does not have. Predictive coding already gives you the first two, as activities and weights. CD is the third. The two are connected. If settling leaves behind errors that are persistent and have structure, that is a sign that a piece of equipment is missing, so PC residuals are a natural signal for the structure search. And we do not have to build a language model from scratch to try this, because all three can live in a cap on top of a transformer that already works.”

Terms and concepts

Temporary activities versus persistent weights. Activities are scratch numbers, recomputed for every input and then thrown away. Weights persist from one input to the next.

IN SYMBOLS. Activities z minimize $E_0(z; x, \theta)$ for the current x . Weights θ change by $\theta \leftarrow \theta - \eta \hat{\nabla}_\theta C$ between inputs.

Prefix; copy reader. The prefix is the text seen so far. A copy reader is a simple memory device that looks back through the prefix for earlier occurrences of the last few characters and proposes whatever followed them. It is the mechanism behind “The captain is Mira.”

Gate. The small learned component that decides how far to trust each reader in the present context. Slide 9 gives its equations.

PC local rule, or BP on the same computation. Two ways to compute the weight update. BP passes an error signal backward through the whole computation. The PC rule compares two settled states and changes each weight using only quantities available at that weight. The slide says “on the same computation” because in the experiments both arms run identical inference, and only the way the weight update is obtained differs.

IN SYMBOLS. BP: $\nabla_\theta L(\text{Readout}(z_\theta^K), y)$ by the chain rule through the K settling steps. PC: $\hat{\nabla}_\theta C = [\partial_\theta E_0(z_\beta) - \partial_\theta E_0(z_0)]/\beta$ (slide 8).

Structural search; refit; “keep what measurably helps”. Propose a change to the wiring, such as adding a reader, removing one, or composing two features. Retrain the affected numbers briefly. Compare with a control that received the same retraining and no structural change. Keep the change only if the deployed predictor is better.

IN SYMBOLS. With candidate P_a , matched control P_{control} and development set A ,

$$U(a) = L_A(P_{\text{control}}) - \min_{\alpha \in [0,1]} L_A((1 - \alpha)P_{\text{control}} + \alpha P_a) \geq 0,$$

and the realized, signed gain is recorded on a separate set. An earlier admission rule required a module of B bytes saving g bpb to repay its storage, which takes $T = 8B/g$ bytes of future text (eight million bytes for 10 kB and 0.01 bpb). That rule rejected nearly everything and was replaced.

A record of past searches. A learned guide scores candidate changes using the outcomes of earlier searches, failures included. It is like a mechanic's notebook. It suggests what to try first and certifies nothing.

IN SYMBOLS. A regressor $\hat{V}_\phi(K, e)$ over descriptors of the candidate template e , the data state K and the history H . Proposals are drawn from $q_t = (1 - \varepsilon_t)q_\phi + \varepsilon_t q_0$ so that exploration never stops.

Persistent structured residuals. Errors that remain after settling and that repeat in a pattern. Random leftover error is noise. Patterned leftover error says the model lacks a feature that would explain the pattern.

IN SYMBOLS. With residual gradient g_a and curvature H_a along a candidate writer, the predicted gain is $\hat{G}_a = \frac{1}{2}g_a^\top (H_a + \lambda I)^{-1}g_a$ (derived on slide 13).

Cap on a frozen transformer. See slide 1. “Frozen” means the base model's weights never change and no gradient flows into it.

Questions you may get

Is structure search just neural architecture search? It shares the outer loop. The differences are that the candidates are small executable readers and features inside one running model, acceptance is by measured gain against a matched control under a storage budget, and the search is meant to run in the background during use.

Slide 8 about 1 minute**The energy view, in familiar terms**

On the slide. Three equations: the free energy E_0 , the nudged energy E_β , and the weak-nudge gradient identity. Three cards: the free phase predicts; the graph is not fixed; a matched BP control.

What to say

“You know this slide already, so I will only point at what is particular to us. First line: a sum of precision-weighted prediction errors plus an evidence term, and the evidence term uses only what is observed at prediction time. Second line: a weak nudge toward the target, used in training only. Third line: in the limit the contrast between the two phases gives the true gradient. Three things are specific to this program. Prediction always uses the free phase, so the target cannot leak into a score. The graph itself can change, because CD proposes new nodes and edges. And our BP control differentiates through the very same settling steps, so when we compare PC with BP, only the learning rule differs.”

Terms and concepts

Nodes, parents, predictions. The network is a graph. Each node holds a vector of activity. Each node also has a small function that guesses its activity from the activities of the nodes feeding into it, which are its parents.

IN SYMBOLS. Node v has activity z_v , parents $\text{pa}(v)$, and prediction $f_v(z_{\text{pa}(v)}; \theta_v)$. The prediction error is $\varepsilon_v = z_v - f_v(z_{\text{pa}(v)}; \theta_v)$.

Precision Λ_v . How seriously each error is taken. A high-precision error is expensive and the network works hard to remove it.

IN SYMBOLS. $\Lambda_v \succeq 0$ and $\|r\|_{\Lambda}^2 = r^\top \Lambda r$. Slide 6 suggests $\Lambda_v = G(z_v)$.

The free energy E_0 ; evidence term. A single number that is small when every node agrees with its parents' guess and with the observed evidence. Settling means sliding downhill on it.

IN SYMBOLS.

$$E_0(z; x, \theta) = \sum_v \frac{1}{2} \|\varepsilon_v\|_{\Lambda_v}^2 + E_{\text{evidence}}(z; x), \quad z^{k+1} = z^k - \eta \nabla_z E_0(z^k).$$

The learning signal is local: $\partial E_0 / \partial \theta_v = -(\partial f_v / \partial \theta_v)^\top \Lambda_v \varepsilon_v$, which involves only node v and its parents.

Free phase and nudged phase; β ; cost C . The free phase is the network's unaided answer. In the nudged phase the correct answer pulls gently on the output, with strength β . Comparing the two settled states shows which local habits were responsible for the error.

IN SYMBOLS. $E_\beta(z) = E_0(z) + \beta C(z, y)$, with C the loss given the target y . Let z_0 and z_β be the corresponding stationary states.

The weak-nudge identity. In the limit of a tiny nudge and exact settling, the difference between the two phases gives exactly the gradient that backpropagation would compute.

IN SYMBOLS. If $z_0(\theta)$ is a smooth stationary branch and $H = \partial_{zz}^2 E_0(z_0)$ is positive definite, then

$$\nabla_{\theta} C(z_0(\theta)) = \lim_{\beta \rightarrow 0} \frac{\partial_{\theta} E_0(z_{\beta}, \theta) - \partial_{\theta} E_0(z_0, \theta)}{\beta}.$$

Sketch: implicit differentiation gives $\partial_{\beta} z_{\beta}|_0 = -H^{-1} \nabla_z C$ and $\partial_{\theta} z_0 = -H^{-1} \partial_{z\theta}^2 E_0$. Expanding the numerator to first order in β gives $-\partial_{\theta z}^2 E_0 H^{-1} \nabla_z C$, which is the chain-rule gradient. A finite β and incomplete settling both introduce error.

“Scores are never contaminated”. The reported loss is computed from the free-phase prediction, which never saw the target. This is a point of experimental hygiene that PC audiences sometimes ask about.

“The graph is not fixed”. CD may add or remove nodes, edges and readers in M . After each change the affected θ_v are refitted, and the result is compared with a control that was refitted without the change.

K settling steps; the matched BP control.

IN SYMBOLS. Both arms predict with $z^K = \text{Settle}_K(E_0, z^0)$, $K = 24$. CD-PC updates θ from the free/nudged contrast with no backward pass. CD-BP uses $\nabla_{\theta} L(\text{Readout}(z_{\theta}^K), y)$, unrolling the same 24 steps. A difference in results can therefore be credited to the learning rule.

Questions you may get

Is this equilibrium propagation or predictive coding? The learning rule is equilibrium-propagation style, applied to an energy of predictive-coding form. The report is careful to say that not every algorithm called PC is equivalent to it.

How do you handle the fact that 24 steps is not equilibrium? From the 24-step free state, an unnudged control and a nudged copy each run 24 further steps, and their difference is used. Residual drift affects both alike and cancels.

Slide 9 about 2 minutes**H2: a small cap on a frozen transformer**

On the slide. Four boxes in a chain: frozen base (2 blocks, width 64, byte-level, 120,576 parameters); 13 readers, each reporting a full next-byte distribution; gate prior $u = Wf(x) + b$ with 1,521 parameters; settle for 24 steps, then mix. Below: the gate energy $E_0(z; x)$ and a sentence explaining $q_{j,s}$, $m = 16$, and the choice of 5 of 13 readers.

What to say

“Here is the working system, and it is deliberately tiny. A two-block transformer predicts text one byte at a time and is frozen. Beside it sit thirteen readers, mostly old-fashioned count models, each of which proposes a complete probability distribution for the next byte. A linear gate looks at the transformer’s hidden state and proposes how far to trust each reader. That proposal is a prior. Before predicting, the gate settles for 24 steps on this energy: stay close to the prior, and also favor whichever readers have been predicting the last sixteen bytes well. So the learned prior proposes trust and recent prediction errors revise it. That is a small, honest PC inference. Finally CD chooses which five of the thirteen readers to keep, by fitting them all and pruning.”

Terms and concepts

Frozen base; blocks; width; byte-level; parameters. A very small transformer. Two layers, each position represented by 64 numbers, vocabulary of the 256 possible byte values, and a context of 64 bytes. It has 120,576 adjustable numbers, all fixed throughout.

IN SYMBOLS. $p_B(y \mid x)$, $y \in \{0, \dots, 255\}$, and a hidden vector $h_B(x) \in \mathbb{R}^{64}$ that the cap may read. No gradient reaches the base.

Reader; suffix-count model. A reader is anything that looks at the prefix and proposes a distribution for the next byte. A suffix-count reader remembers, for each short string seen in training, how often each byte followed it. If the current context was never seen, it falls back to a shorter one.

IN SYMBOLS. For a stored context s with shortened context s' ,

$$q_k(y \mid s) = \frac{n(s, y) + \alpha q_{k-1}(y \mid s')}{n(s) + \alpha},$$

with counts n and backoff strength α . Eight such readers differ in depth $k \in \{1, 2, 3, 4, 6, 8\}$ and in $\alpha \in \{4, 16\}$. The shared dictionary is built from 700,000 bytes of training text and then frozen. It holds 89,746 contexts and occupies about 22 MiB. It is a large part of the system and has to be counted as such.

Copy reader. See slide 7. Four variants differ in how many characters must match and in what they fall back to.

“Each reports a full next-byte distribution”. This was the main change from the unsuccessful first cap, which squeezed each reader into a few learned features. Here each reader says exactly which bytes it supports and with what probability. It is the difference between a flag saying “something relevant was found” and a list of candidate answers with their weights.

IN SYMBOLS. $q_j(\cdot | x) \in \Delta^{255}$ for $j = 0, \dots, 12$, with $q_0 = p_B$, all sharing a uniform floor so that no byte has probability zero.

Gate prior $u = Wf(x) + b$; **1,521 parameters.** A single linear layer. Its input is the base's hidden vector together with four summary numbers for each reader: how spread out its distribution is, how confident its top guess is, how much data stands behind it, and whether it agrees with the base.

IN SYMBOLS. $f(x) \in \mathbb{R}^{116}$, since $64 + 4 \times 13 = 116$. $W \in \mathbb{R}^{13 \times 116}$ and $b \in \mathbb{R}^{13}$, giving $13 \times 116 + 13 = 1,521$ parameters. The vector $u_\theta(x) \in \mathbb{R}^{13}$ holds trust logits.

Softmax; mixture. Softmax turns any list of scores into positive weights that sum to one. The final prediction is the weighted blend of the readers' distributions.

IN SYMBOLS. $w_j(z) = e^{z_j} / \sum_k e^{z_k}$ and $p(y | x) = \sum_{j \in \mathcal{A}} w_j(z^K) q_j(y | x)$ over the active set $\mathcal{A}$.

The gate energy; $q_{j,s}$; m ; λ . The first term says “stay near what the learned prior proposed.” The second says “prefer weightings under which the last sixteen bytes, already seen, would have been well predicted.” The number $q_{j,s}$ is the probability reader j gave, before seeing it, to the byte that then occurred s steps ago.

IN SYMBOLS.

$$E_0(z; x) = \frac{1}{2} \|z - u_\theta(x)\|^2 + \lambda \frac{1}{m} \sum_{s=1}^m \left[-\log \sum_{j \in \mathcal{A}} w_j(z) q_{j,s} \right], \quad m = 16, \lambda = 2.$$

Read as probabilities, e^{-E_0} is a unit Gaussian prior on the logits centered at $u_\theta(x)$, times a tempered likelihood of the recent bytes under the mixture. Settling is therefore approximate MAP inference of the trust logits. The gradient is

$$\nabla_z E_0 = (z - u_\theta) + \frac{\lambda}{m} \sum_{s=1}^m (w(z) - r_s(z)), \quad r_{j,s} = \frac{w_j q_{j,s}}{\sum_k w_k q_{k,s}},$$

where r_s is the posterior responsibility of each reader for byte s . Settling pulls the weights toward the average responsibility, held back by the prior. The step size is 0.3 and $K = 24$.

How the gate learns.

IN SYMBOLS. The nudged energy is $E_\beta = E_0 + \beta [-\log \sum_j w_j(z) q_j(y | x)]$ with $\beta = 0.05$. Since $\partial E_0 / \partial u = -(z - u)$, the identity of slide 8 gives $\nabla_u C \approx (z_f - z_\beta) / \beta =: d$, hence $\widehat{\nabla}_W C = d f(x)^\top$ and $\widehat{\nabla}_b C = d$. These are averaged over a minibatch of 256 and passed to AdamW.

“CD chooses which 5 of the 13”; compression-first search. Start with all thirteen readers, then repeatedly try removing one, retrain briefly, compare with a control, and keep the best removal, until five remain.

IN SYMBOLS. 240 initial updates, then eight rounds each with two candidate removals and one control at 24 updates apiece, then 240 final updates: $240 + 8 \times 3 \times 24 + 240 = 1,056$ updates, which is the budget that the “matched” controls on slide 10 also receive.

Questions you may get

Is the settling a real hierarchy? No. It is one layer of latent trust logits with a Gaussian prior and an evidence term. That is why the next slides call the PC effects small, and why the first open problem on slide 13 is about richer graphs.

Is the dictionary cheating? It is extra memory and extra data use, and the results slide attributes most of the gain to it. It never sees test text, and it never sees the byte being predicted.

Slide 10 about 1.5 minutes**Where the gain comes from**

On the slide. Large figure: $2.90 \rightarrow 2.34$ bits per byte on held-out Shakespeare, five-seed mean. A table of five conditions with their losses (2.8962, 2.4285, 2.3460, 2.3357, 2.3323) and what each comparison isolates.

What to say

“The headline is a drop from 2.90 to 2.34 bits per byte, which is large for a cap this small. The table is there to stop anyone, me included, from crediting that to the wrong component. Read it top to bottom. Simply blending all thirteen readers with one fixed set of weights, with no gate and no PC, already gets to 2.43. That is more than four fifths of the whole gain, and it comes from the richer memory. A learned, context-sensitive gate over five hand-picked readers takes it to 2.346. Letting CD choose the five readers, with the same training budget, gains another hundredth of a bit, in every one of five seeds. Using all thirteen is slightly better still, so the compression costs about three thousandths of a bit.”

Terms and concepts

Bits per byte (bpb). A score for a predictor. Each time, look at the probability it gave to the byte that did come next. High probability costs few bits, low probability costs many. Average over the text. A model that knows nothing about 256 possible bytes scores 8.

IN SYMBOLS. $\text{bpb} = -\frac{1}{N} \sum_{t=1}^N \log_2 p(y_t | x_{<t})$. Perplexity is 2^{bpb} : the base’s 2.896 corresponds to choosing among about 7.4 equally likely bytes, and 2.336 to about 5.0. A predictor with loss L can in principle compress the text to L bits per byte, which is the link to the “compression” in CD.

Held-out; five-seed mean. Held-out text was never used for fitting or for choosing settings. A seed is the random starting point of a run. Five runs that differ only in their seed show that a result is not one lucky run.

IN SYMBOLS. Scores are on 32,768 test targets of Tiny Shakespeare. A following segment of 22,938 targets gives the same ordering. The seeds share the frozen base, the corpus and the dictionary, so they are not independent replications with fresh data. For scale, five out of five differences with the same sign has probability $2^{-5} = 1/32$ under a fair coin.

Global mixture of all 13 readers, no gate. One fixed set of thirteen weights, used for every context.

IN SYMBOLS. $p = \sum_j w_j q_j$ with constant w , fitted by expectation–maximization: $r_{ij} = w_j q_j(y_i | x_i) / \sum_k w_k q_k(y_i | x_i)$, then $w_j \leftarrow \frac{1}{N} \sum_i r_{ij}$. Loss 2.428546. Its gain over the base, 0.4677, is 83% of the total gain of 0.5605.

Fixed five readers, PC, 1,056 updates. Five readers chosen by hand in advance, with a gate trained by the PC rule for the same number of updates as the structure search uses. This is the matched control for the next row.

IN SYMBOLS. Loss 2.345958. The gate adds 0.0826 over the global mixture, with fewer readers.

Compression-first CD–PC. The procedure of slide 9: fit all, prune to five, by measured loss.

IN SYMBOLS. Loss 2.335741. Against the matched fixed-five control the gain is 0.010217 bpb, positive in 5 of 5 seeds on this segment and on the following one (0.005911).

All 13 readers, PC; “the accuracy reference”. No pruning. It shows what compression costs.

IN SYMBOLS. Loss 2.332323, better than the pruned cap by 0.003418 bpb. The reference implementation does not yet turn “five active readers” into a measured saving of time or storage.

Questions you may get

How does this compare with a plain adapter of the same size? That comparison has not been run at this stage, and it is the acceptance test for the next stage (slide 14). Earlier prototypes were compared with small neural caps and were competitive, under different conditions.

Is Tiny Shakespeare overused here? Yes. It guided all development. Replication on fresh corpora is the first item in the plan.

Slide 11 about 1.5 minutes**What predictive coding did, and did not do**

On the slide. Three figures: 0.0008 (CD-BP beat CD-PC by this much); +0.0025 (loss when settling is switched off, 5 of 5 seeds; wrong-example evidence costs +0.0023); +0.0048 (extra forgetting with settling off, 0.0660 $\rightarrow$ 0.0708). Bottom line: PC earns its place as inference so far.

What to say

“This is the slide I most want to get right in front of this audience. As a learning rule, PC tied with backprop. The matched BP arm was ahead by eight ten-thousandths of a bit, which is nothing, and the PC updates track BP gradients more closely as you shrink the nudge and settle longer, as the theory says they should. So I cannot tell you PC learns better. What I can tell you is that the settling step does something. Switch it off at prediction time and the loss goes up by two and a half thousandths of a bit, in every seed. Leave it on but feed it another example’s recent history, and you lose almost the same amount, which shows that the benefit comes from the evidence and not from the extra computation. Settling also gives some protection against forgetting. These are small numbers from a gate with fifteen hundred parameters and one latent layer. Whether they grow when the graph is richer is an open question, and it is your kind of question.”

Terms and concepts

“The matched CD-BP arm beat CD-PC by 0.0008”.

IN SYMBOLS. Compression-first CD-BP 2.334918 against CD-PC 2.335741, a difference of 0.000823 bpb (0.000858 on the following segment). Both arms use identical readers, search, budget and prediction-time settling.

“Updates track BP gradients as β shrinks and settling lengthens”. A sanity check on the implementation. The PC update is compared, as a vector, with the gradient BP computes at the same point. The agreement improves in the direction the theorem of slide 8 predicts.

IN SYMBOLS. The error of the finite-difference estimate is $O(\beta)$ plus a term that depends on the distance of z^K from the true stationary point.

“Settling switched off at prediction time”. Take a trained gate, and predict directly from the prior $u_\theta(x)$ without running the 24 steps.

IN SYMBOLS. Replace $w(z^K)$ by $w(u_\theta(x))$. The loss rises by 0.002484 bpb on the original segment, in 5 of 5 seeds, and by 0.001070 on the following segment. In a separate experiment, gates retrained with and without inference differ by 0.001703 and 0.000663, again in all seeds.

“Another example’s recent evidence”. A negative control. The settling runs as usual, but the record of how the readers performed on the last sixteen bytes is taken from a different, unrelated position in the text. If the benefit of settling came merely from doing more computation, this would help as much as the real thing.

IN SYMBOLS. Replace $\{q_{j,s}\}_{s \leq m}$ by those of a permuted example. The loss rises by 0.002305 bpb, nearly the whole settling benefit.

“Extra forgetting with settling off”.

IN SYMBOLS. With the same checkpoints before and after adaptation to *Alice in Wonderland*, the forgetting F_A of slide 12 is 0.065971 with settling and 0.070766 without, a difference of 0.0048. For Python source the figures are 0.079041 and 0.083149. This changes inference only and is not a retrained model without PC.

“Real hierarchies”. The H2 gate has one layer of latent variables. A PC network in the usual sense has several, with errors passed between levels. The slide’s closing sentence asks whether the inference benefit grows in that setting.

Questions you may get

What happened with the first cap? It had a nonlinear PC cap with 32,064 parameters and a learned feature summary of each reader. A plain fixed BP cap beat CD-PC in all three seeds, and PC training took three to four times as long on CPU. Two lessons were drawn: give the cap richer evidence, and give settling a job of its own.

Are these effects significant? They are small and they have the same sign in every seed and on both segments. The seeds share a base and a corpus, so the report describes them as consistent, and does not claim an independent replication.

Slide 12 about 1 minute**Forgetting under domain shift**

On the slide. Four bars: share of the cap's Shakespeare advantage erased after 1,024 updates on a new domain. Alice, PC: 11.8%. Alice, PC with 10% replay: 6.9%. Python, PC: 14.1%. Python, PC with 10% replay: 8.8%. Two cards: plasticity is kept (97.7% and 99.3% of the new-domain gain); PC and BP curves coincide. Bottom line: the library kept its books while the librarian picked up new habits.

What to say

“We took the trained cap and kept training it on something else, first *Alice in Wonderland* and then Python code, and asked how much Shakespeare it lost. It lost twelve to fourteen percent of its advantage. That is moderate: far from a collapse, and clearly not immunity. Mixing one old example into every ten during adaptation cuts the loss by about forty percent and costs almost nothing in new learning. PC and BP are indistinguishable here. What I find interesting is how the forgetting happens. All the memory was frozen. Every count in the dictionary is unchanged. What moved was the gate, the policy for whom to trust. The library still has all its books, and the librarian has picked up new habits about which shelf to check. So what needs protecting in a system like this is access.”

Terms and concepts

Domain shift. A change in the kind of data. Here, from Shakespeare to a Victorian children's novel, or to Python source code.

“Share of the advantage erased”. Before adaptation the cap beats the base on Shakespeare by some margin. Afterwards the margin is smaller. The bar shows what fraction of it has gone.

IN SYMBOLS. With F_A as on slide 2,

$$E_A = \frac{F_A}{L_A(\text{frozen base}) - L_A(\theta_A)} = \frac{F_A}{2.896241 - 2.335741}.$$

For Alice with PC, $F_A = 0.065971$ and $E_A = 11.8\%$. For Python, $F_A = 0.079041$ and $E_A = 14.1\%$. In perplexity terms the old-task loss worsens by factors $2^{F_A} \approx 1.047$ and 1.056 .

1,024 updates. The fixed adaptation budget. Each update uses a minibatch of 256 examples from the new domain. Only the gate is trainable. The base, the dictionary, the chosen readers and the feature normalization stay frozen.

Replay. While learning the new material, keep practising a little of the old.

IN SYMBOLS. 26 of the 256 examples in each minibatch (10.16%) are drawn from a fixed buffer of 1,024 old *training* examples. Forgetting falls to 0.038921 (Alice) and 0.049448 (Python), reductions of 41.0% and 37.4%.

Plasticity is kept. Plasticity is the ability to learn the new thing. A method that prevents forgetting by preventing learning would be useless, so both are measured.

IN SYMBOLS. The new-domain gain G_B is 0.087650 without replay and 0.085625 with it for Alice (97.7% retained), and 0.251409 against 0.249538 for Python (99.3%).

PC and BP curves coincide.

IN SYMBOLS. For Alice, F_A is 0.065971 under PC and 0.066171 under BP. For Python, 0.079041 and 0.079625.

Available versus accessible.

IN SYMBOLS. In $p = \sum_j w_j(x; \theta) q_j$, the q_j are frozen and only $w_j(\cdot; \theta)$ changes. Forgetting here is entirely a drift in the routing policy.

Questions you may get

Could it simply be the optimizer running longer? A control that continued training on Shakespeare alone changed the loss by -0.0013 bpb. The effect is interference from the new domain.

Can anything be guaranteed? Yes, for retained complete predictors. A fixed mixture $\rho p_{\text{old}} + (1 - \rho) p_{\text{new}}$ loses at most $-\log_2 \rho$ bits per target relative to the old predictor, and a Bayesian mixture over retained branches loses at most $-\log_2 \pi_j$ bits in total. H2's learned gate is not such a mixture, so neither bound applies to this experiment.

Slide 13 about 1.5 minutes**Open problems where PC expertise counts**

On the slide. Six cards. Top row: caps as summed energies; precision from later use; residuals guide search. Bottom row: affordable settling; locality and retention; learning while serving.

What to say

“Here is what I would like help with. The top row is about the energy itself. If several caps each contribute an energy term, does the sum have a unique minimum, and does the answer depend on the order in which they settle? Can the precisions be set by the Fisher matrix of downstream predictions? Can the residuals that settling leaves behind tell us what structure to add next? The bottom row is engineering. Settling has to become cheap. We need to find out whether local learning makes it easier to grow modules without disturbing old ones. And we want all of this to run while the system is serving predictions. None of these six has an experiment behind it yet.”

Pick the two that best fit the people in the room and say a sentence more about each.

Terms and concepts

Caps as summed energies. Several specialist modules share one workspace. Each says how unhappy it is with the current state, and the workspace settles to the state that makes them least unhappy in total. Good specialists do not automatically make a good team: two may count the same evidence twice, or pull in opposite directions.

IN SYMBOLS. For an active set A of modules, $E_A(z; x) = E_B(z; x) + \sum_{j \in A} \alpha_j E_j(z; R_j h_B(x), M_j)$. If every term is convex in z the minimum is unique and independent of order. For nonconvex terms, block-coordinate settling can reach different stationary points in different orders. A training objective aimed at composition samples subsets and orders: $\mathbb{E}_{A, \pi, (x, y)} [-\log p_{A, \pi}(y | x)] + \lambda \mathcal{R}_{\text{old}} + \eta \mathcal{R}_{\text{duplicate}} + \xi \mathcal{R}_{\text{order}}$.

Precision from later use. See slide 6. Let the energy care most about the directions of state space that change downstream behavior.

IN SYMBOLS. Set $\Lambda_v = G(z_v)$, or a low-rank or diagonal approximation of it, in E_0 . The open questions are how to estimate G cheaply and whether it should be held fixed during settling.

Residuals guide search. If a certain kind of error keeps coming back after settling, some feature that would explain it is missing. A cheap calculation estimates how much a candidate feature would help before anything is trained.

IN SYMBOLS. Suppose adding a candidate with coefficient u changes the loss by $\Delta L(u) \approx g^\top u + \frac{1}{2} u^\top H u$. With a ridge penalty, minimize $g^\top u + \frac{1}{2} u^\top (H + \lambda I) u$: the minimizer is $u^* = -(H + \lambda I)^{-1} g$ and the predicted gain is

$$\hat{G} = \frac{1}{2} g^\top (H + \lambda I)^{-1} g.$$

This is the exact optimum of the quadratic surrogate and no more. Candidates are ranked by it, and the best are then refitted against a matched control.

Affordable settling. Twenty-four extra steps per prediction is tolerable for a tiny gate and would not be for a large model.

IN SYMBOLS. Options: warm-start z^0 from the previous position; train a small network $a_\phi(x)$ to predict the settled state by minimizing $\|a_\phi(x) - z^K\|^2$ and settle only a few steps from there; stop when $\|\nabla_z E_0\| < \text{tol}$; settle only when the prior is uncertain. In the first cap, PC training took 5.65 s against 1.57 s for BP on the fixed architecture and 29.0 s against 8.7 s with structure search, on CPU.

Locality and retention. If each weight update depends only on nearby quantities, it may be easier to add a new module and train it without disturbing the old ones. That is a hope. So far only replay has been measured.

IN SYMBOLS. A usable contract is near-commutation of updates. For update maps U, V with $d(UV(z), VU(z)) \leq \eta$, a later L -Lipschitz computation sees a difference of at most $L\eta$ between the two orders. The contract has to be measured in a behavioral metric, and it does not follow merely from using PC.

Learning while serving; atomic version publishing. Keep answering with the current version while a copy learns in the background, and switch to the new version in a single step once it has passed its tests. A prediction in progress never sees a half-updated model.

IN SYMBOLS. Fast loop: predict with pinned (M_t, θ_t) . Middle loop: bounded updates of θ on a learner copy, with retention data. Slow loop: structure search and guide training. Publish (M_{t+1}, θ_{t+1}) atomically, and keep the means to roll back.

Questions you may get

Which of these would you do first? Affordable settling and the summed-energy question, since both block the next scale of experiment.

Slide 14 about 30 seconds**Close**

On the slide. Headline: “A learner that can preserve, reach, revise and share structure while it runs.” Four steps: Now, Next, Then, Later. Last line: the full report is available.

What to say

“That is the aim: a learner that can preserve what it knows, reach it when needed, revise it, and share it, all while running. The plan is staged, and each stage has a test it must pass before we spend more. First, replicate on fresh data. Then a base model of tens of millions of parameters, where the cap has to beat a strong ordinary adapter at equal memory and compute. Then a billion-parameter open model with several caps working together. Only after that, exchange of modules between machines, and a frontier-scale base. I would be glad to talk with anyone who wants to take on one of the problems on the previous slide.”

Terms and concepts

Preserve, reach, revise, share. These four verbs correspond to the goals of slide 2: retention; access, which slide 12 showed to be a separate matter from retention; the three kinds of revision of slide 7; and exchange of modules.

Fresh small corpora; growing readers; retained gates; memory ceilings; task streams. The immediate work. Use text that played no part in development. Let the reader memory grow during use. Keep a copy of the old gate alongside the new one as one form of protection. Fix a total memory budget so that “remember everything” is not an available answer. Test on long sequences of domains, with returns to earlier ones.

A strong ordinary adapter at matched memory and compute. An adapter is a small trainable add-on to a frozen model, such as LoRA. The cap has to beat one when both are given the same memory and the same computation. If it does not, the extra machinery is not paying for itself.

IN SYMBOLS. A budget identity keeps the comparison honest. With base cost C_B per token, cap fast-path cost C_C , K candidate trials of cost C_T every B tokens, and background work C_H over T tokens, the average cost per token is $\overline{C} = C_B + C_C + KC_T/B + C_H/T$. If a search costs $2000 C_B$, the fast path $0.02 C_B$ and background work $0.01 C_B$, then an overhead ceiling of 10% allows one search per $2000/0.07 \approx 28,600$ tokens. Structure search therefore has to be an occasional investment whose results are reused.

Composable caps; symbolic readers; held-out compositions. Several caps active together (the first card of slide 13). Readers whose distribution comes from a reasoner or a program, for instance a PLN query or a MeTTa function. Tests in which the required combination of operations never appeared in training (slide 2).

Module exchange between workers with different data. The decentralized goal. Each worker discovers modules on its own data, publishes them, and tests what it receives before adopting it.

IN SYMBOLS. Measured by predictive transfer per communicated byte, and by the damage done by stale or incompatible modules, under data that are not identically distributed across workers.

Frontier-base cap; standalone GT/CD learner. Two different end points. A cap on a frontier model would show practical value and would not show that CD can replace pretraining. A standalone learner would need a competitive scaling curve of its own, at several model sizes, and is attempted only if the smaller stages succeed.

Questions you may get

Is the code available? What exists: the report's evidence bundle, which reproduces every table from saved analysis files, and the experiment archives, which hold the training code, checkpoints and corpora. Decide before the talk what you are willing to share, and with whom.

What would make you abandon the PC part? A fair answer: if, at the next scale and with richer graphs, settling still adds only thousandths of a bit and costs a multiple of the compute, the inference step would be kept only in amortized form, and BP would remain the learning rule.